Subject:
Year. Month. Date. ( )

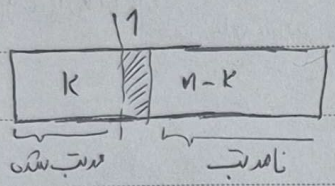
DS

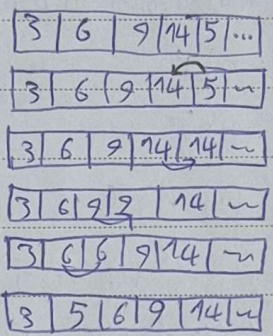
معیار های ارزیابی الگوریتم : ۱- زمان اجرا ۲- حافظه مصرفی ۳- پیچیدگی کد (خوانایی کد)
۴- امنیت ۵- تعمیم پذیری کد

عوامل موثر بر زمان اجرا (۱) : ۱- سخت افزار ۲- خود الگوریتم ۳- اندازه و ماهیت ورودی ۴- زبان برنامه نویسی
۵- سیستم عامل

Insertion Sort (درجی)

- با یک زیرمجموعه به اندازه ۱ شروع کن
- هر بار عضو بعدی از عناصر مرتب نشده را به زیرمجموعه مرتب شده اضافه کن (insert)





مرتب

cost	x times executed	
c_1	n	for k=2 to Length(A)
c_2	$n-1$	key = A[k]
c_3	$n-1$	i = k-1
c_4	$\sum_{k=2}^{n} t_k$	while(i>0 and A[i]>key)
c_5	$\sum_{k=2}^{n} (t_k-1)$	A[i+1] = A[i]
c_6	$\sum_{k=2}^{n} (t_k-1)$	i = i-1
c_7	$n-1$	A[i+1] = key

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4\sum_{k=2}^{n} t_k + c_5\sum_{k=2}^{n} (t_k-1) + c_6\sum_{k=2}^{n} (t_k-1) + c_7(n-1)$$

$$T(n) = c_1 n + (c_2+c_3+c_7)(n-1) + c_4\sum_{k=2}^{n} t_k + (c_5+c_6)\sum_{k=2}^{n} (t_k-1)$$

بهترین حالت $t_k = 1$

$$T(n) = c_1 n + (c_2+c_3+c_7)(n-1) + c_4(n-1) = c_1 n + (c_2+c_3+c_4+c_7)(n-1)$$

$$\boxed{T(n) = A_1 n + B_1}$$

بدترین حالت $t_k = k$

$$\sum_{k=2}^{n} t_k = \sum_{k=2}^{n} k = \frac{n(n-1)}{2} - 1$$

$$\sum_{k=2}^{n} (t_k-1) = \sum_{k=2}^{n} (k-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

$$T(n) = c_1 n + (c_2+c_3+c_7)(n-1) + c_4\left(\frac{n(n-1)}{2} - 1\right) + (c_5+c_6)\left(\frac{n(n-1)}{2}\right)$$

$$T(n) = A_2 n^2 + B_2 n + C_2$$

حالت متوسط : ترکیب های مختلف ممکن (امید ریاضی) $\left(t_k = \frac{k+1}{2}\right)$

$$T(n) = A\sum_{k=1}^{n} \bar{t}_k + Bn + C = A\sum_{k=1}^{n}\left(\frac{1}{k}\sum_{i=1}^{k} i\right) + Bn + C$$

$$\boxed{T(n) = A_3 n^2 + B_3 n + C_3}$$

$A_2 > A_3$ ولی در مورد تغییر ضرایب نمی‌توان نظر داد.

P4PCO

Algorithm Analysis: Big oh, Omega, Teta

① Order (تخمین بالا (سقف
یعنی وقتی n^2 باشه حداکثر تا n^2 زمان داریم.

② Ω (کف) تخمین پایین
یعنی وقتی n^2 باشه حداقل تا n^2 زمان داریم.

③ Θ تخمین دقیق تا
$\Theta(n^2)$ یعنی درجه دقیقا درجه 2 است.

Algorithm Analysis: Big Oh

تابع $f(n)$، $O(g(n))$ است اگر ثابت‌های n_0 و c وجود داشته باشند به طوری که برای تمام $n \geq n_0$ داشته باشیم: $f(n) \leq cg(n)$

به عبارتی، برای تابع $g(n)$، $O(g(n))$ مجموعه‌ای از توابع است:

$$O(g(n)) = \{f(n): \exists c, n_0 > 0 \text{ such that } \forall n \geq n_0 \quad 0 \leq f(n) \leq cg(n)\}$$

به زبان ساده $cg(n)$ بازای همای بزرگ، یک حد بالایی برای $f(n)$ است.

$3n+2 = O(n) \rightarrow 3n+2 \leq 4n$ for $n \geq 2$

$3n+3 = O(n) \rightarrow 3n+3 \leq 4n$ for $n \geq 3$

$100n+6 = O(n) \rightarrow 100n+6 \leq 107n$ for $n \geq 10$

$10n^2+4n+2 = O(n^2) \rightarrow 10n^2+4n+2 \leq 11n^2$ for $n \geq 5$

$6 \times 2^n + n^2 = O(2^n) \rightarrow 6 \times 2^n + n^2 \leq 7 \times 2^n$ for $n \geq 4$

Algorithm Analysis: Omega

$0 \leq cg(n) \leq f(n)$ دقیقا مثل order است ولی

$2n+10 = \Omega(n) \rightarrow 2n+10 \geq 2n$ for $n \geq 1$

$3n^2+2n+20 = \Omega(n^2) \rightarrow 3n^2+2n+20 \geq 3n^2$ for $n \geq 1$

$3n^2+2n+20 = \Omega(n)$ an^2+bn+c is $\Omega(n^2)$

Algorithm Analysis: Theta

تابع $f(n)$، $f(n)$ $\Theta(g(n))$ است اگر ثابت‌های $c_1, c_2, n_0 > 0$ وجود داشته باشند به طوری که برای تمام $n \geq n_0$ داشته باشیم:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$f(n)$ از مرتبه $\Theta(g(n))$ است اگر هم $O(g(n))$ و هم $\Omega(g(n))$ باشد.

Algorithm Analysis: Little oh

(همون order است و به‌جای کوچکتر، الزاماً کوچکتر)

$$o(g(n)) = \{f(n): \forall c > 0 \ \exists n_0 > 0 \text{ such that } \forall n \geq n_0 \quad 0 \leq f(n) < cg(n)\}$$

کوچکتر یا برابر ← الزاماً کوچکتر

$$\omega(g(n)) = \{f(n): \forall c > 0 \ \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0 \quad 0 \leq cg(n) < f(n)\}$$

الزاماً کوچکتر ↑

Algorithm Analysis: little Omega

Subject:
Year. Month. Date. ( )

صحی

if $f(n)=\Theta(g(n))$ and $g(n)=\Theta(h(n))$ then — خاصیت تعدی (transitivity)

$f(n)=\Theta(h(n))$ ← $f(n)=O(g(n))$ ~ ~ $O(g(n))$ ~

مثال خود مونی ← $A\subseteq B$, $B\subseteq C$ ⟶ $A\subseteq C$

خاصیت بازتابی (reflexivity)

$f(n)=\Theta(f(n))$, $f(n)=O(f(n))$, $f(n)=\Omega(f(n))$

خاصیت تقارنی (symmetry)

$f(n)=\Theta(g(n))$ if and only if $g(n)=\Theta(f(n))$

تقارن ترانهاده (transpose symmetry)

$f(n)=O(g(n))$ if and only if $g(n)=\Omega(f(n))$

اگر $f(n)=O(kg(n))$ برای ثابت $k>0$، آنگاه $f(n)=O(g(n))$ منظورش اینه که اوردر $100n=20n=n$

اگر $f_1(n)=O(g_1(n))$, $f_2(n)=O(g_2(n))$

آنگاه $(f_1+f_2)(n)=O(\max(g_1(n),g_2(n)))$

اگر $f_1(n)=O(g_1(n))$, $f_2(n)=O(g_2(n))$

آنگاه $f_1(n)f_2(n)=O(g_1(n)g_2(n))$

قیاس (analogy) ← Runtime Analysis

$f(n)=O(g(n)) \approx a\le b$

$f(n)=\Omega(g(n)) \approx a\ge b$

$f(n)=\Theta(g(n)) \approx a=b$

$f(n)=o(g(n)) \approx a<b$

$f(n)=\omega(g(n)) \approx a>b$

قواعد مفید:

- جملات ساده (read, write, assign): $O(1)$ (ثابت)
- عملیات ساده ($+,-,*,/,==,>,<,>=,<=$): $O(1)$ (ثابت)
- ترکیبی از جملات ساده و/یا عملیات ساده: $O(1)$ (قاعده جمع)
- هزینه برنامه کدبیت سرهم اجرا می‌شود: پرهزینه‌ترین مستمت
- حلقه‌ها (for, do, while, ...): هزینه بدنه حلقه × تعداد تکرار حلقه
- جملات شرطی (if (cond) then body1 else body2 endif): $T(n)=O(\max(T_1(n),T_2(n)))$

Subject: 40
Year. Month. Date. ( )

slides ویدئو ۳ (25-37)

max subsequence Problem (محاسبه بیشترین حاصل جمع ترتیبی)

مثال: $-2, 11, -4, 13, -5, -2$

جواب ← $(A_2 + A_3 + A_4) = 20$

۴ الگوریتم مختلف:
- $O(n^3)$ ← بدیهی (همه حالات را حساب می‌کنیم)
- $O(n^2)$ ← به جای یک for اضافه هر مرحله عنصر $a[i]$ را اضافه می‌کنیم.
- $O(n \lg n)$ ← تقسیم و حل (بازگشتی) جواب = (استدلال بازه‌ی راست، بازه‌ی چپ، بازه‌ی وسط) Max
- سوپر * $O(n)$ ← بهترین راه:

```
int maxSum = 0, thisSum = 0;
for(int i=0; i<a.size(); j++) {
    thisSum += a[i];
    if(thisSum > maxSum) maxSum = thisSum;
    else if(thisSum < 0) thisSum = 0;
}
return maxSum;
```

- ترتیب اعداد مورد جستجو با عدد منفی شروع یا تمام نمی‌شود.
- ترتیب اعداد مورد جستجو نمی‌تواند پیشوندی یا مجموع کمتر از صفر داشته باشد.

مثال: $-2, 11, -4, 13, -5, -2$

نباید این اگر مجموع A_j ~ A_i کمتر از صفر شود، می‌توان i را تا $j+1$ این جلو برد (و this sum را صفر کرد)

تحلیل الگوریتم‌های بازگشتی (recursive)

- روش‌های حل رابطه‌های بازگشتی: ① حدس و اثبات (استقرا) ② تکرار یا جایگذاری ③ درخت بازگشتی ④ قضیه اصلی

مثال ← mergeSort (مرتب‌سازی ادغامی) $O(n \log n)$

```
mergeSort(A[i..j])
if(i<j){
    mid = (i+j)/2
    mergeSort(A[i..mid]);
    mergeSort(A[mid+1..j]);
    merge(A[i..mid], A[mid+1..j]);
}
```

$\longrightarrow T(n) = 2T(n/2) + n$

باید ثابت کنیم ثابت c وجود دارد که $T(n) \le cn \log n$

$T(n) = 2T(\lfloor n/2 \rfloor) + n \le 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n$ ← استقرایی نیم

$T(n) \le cn \log(n/2) + n = cn \log(n) - cn \lg(2) + n$

$\le cn \log n - cn + n \le cn \log n \rightarrow T(n) \le cn \lg n$

برای $n \ge 1$

O(?)

مثال مهم (تغییر متغیر)

$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n \longrightarrow n = 2^m$

$T(2^m) = 2T(2^{m/2}) + m$ ← تغییر تابع $S(m) = T(2^m)$

$\rightarrow S(m) = 2S(m/2) + m$ ← mergesort $S(m) = O(m \lg m)$

$\rightarrow T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$

Subject: 38

Year. Month. Date. ( )

روش تکرار با جایگذاری رابطه بازگشتی (روش دوم)

$T(n) = 2T(n/2) + n \quad , \quad T(1) = 1$

★ رابطه بازگشتی تا رسیدن به جواب ساده ادامه می‌دهد.

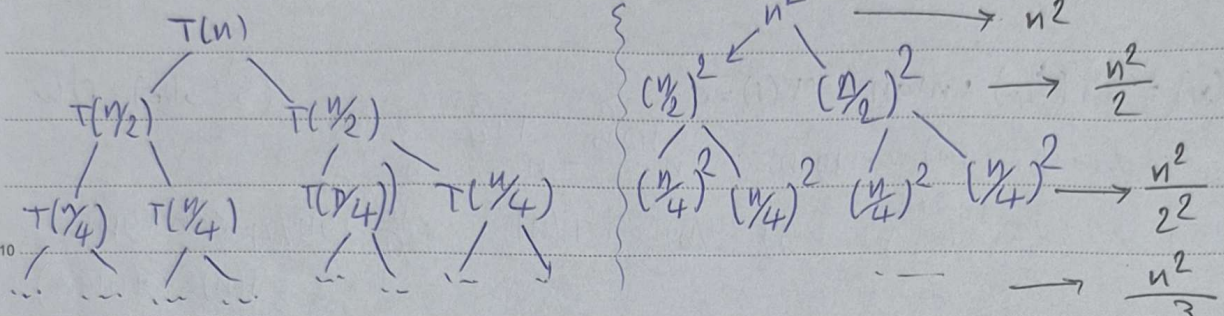
$T(n/2) = 2T(n/4) + n/2 \rightarrow T(n) = 2(2T(n/4) + n/2) + n = 4T(n/4) + n + n$

$= 4(2T(n/8) + n/4) + 2n = 8T(n/8) + 3n = 2^k T(1) + kn$

$= n + n\log n$

$T(n) = 2T(n/2) + n^2 \quad , \quad T(1) = 1$

روش درخت بازگشتی:



$T(n) = n^2\left(1 + \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \dots + \frac{1}{2^{\log n}}\right) = \Theta(n^2)$ هندسی کاهشی

نمونه سوال: $n2^{\sqrt{2\log n}} \in O(n^3)$ برقرار است یا نه؟

باید بررسی کنیم به ازای $n = \infty$ رابطه $n2^{\sqrt{2\log n}} \geq n^3$ برقرار است یا نه

$\ln n + \sqrt{2\log n}\log 2 \geq 3\log n \rightarrow c\sqrt{2\log n} \geq c\log n$

برقراریت ✗

نکته: استرلینگ $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

$\lg n! = \lg\sqrt{2\pi n} + n\lg(n/e) \rightarrow \frac{1}{2}\lg 2\pi + \frac{1}{2}\lg n + n\lg n - n\lg e$

$\rightarrow \frac{1}{2}\lg n + n\lg n - n = \Theta(n\lg n) \rightarrow \boxed{\log n! \simeq \Theta(n\log n) \simeq \log n^n}$

اینا رو مرتب کن ← (کلا واسه مقایسه‌ها وقتی میگیم حل می‌شود)

$\log n! \;,\; n^{\log\log n} \;,\; n^2 \;,\; 2^n \;,\; e^n \;,\; n2^n$

① ③ ② ③ ⑤ ④

قضیه اصلی: برای رابطه بازگشتی $T(n) = aT(n/b) + f(n)$ که $a \geq 1$, $b > 1$ داریم:

① - اگر $f(n) = O(n^{\log_b a - \varepsilon})$ برای $\varepsilon > 0$ ، $T(n) = \Theta(n^{\log_b a})$

② - اگر $f(n) = \Theta(n^{\log_b a})$ ، آنگاه $T(n) = \Theta(n^{\log_b a}\lg n)$

③ اگر $f(n) = \Omega(n^{\log_b a + \varepsilon})$ برای $\varepsilon > 0$ و اگر $af(n/b) \leq cf(n)$ برای $c < 1$ و n های به اندازه کافی بزرگ، آنگاه $T(n) = \Theta(f(n))$

PAPCO

Subject: جلسه 6

Year. Month. Date. ( )

$T(n) = 9T(n/3) + n \quad T(1) = c$

$a = 9, b = 3, f(n) = n$

مثال: (حالت 1)

$n^{\log_b^a} = n^2 \quad n = O(n^{2-\varepsilon}) \rightarrow T(n) = \Theta(n^2)$

مثال: (حالت 2)

$T(n) = T(2n/3) + 1 \quad T(1) = c$

$a = 1, b = 3/2, f(n) = 1 \quad n^{\log_b^a} = n^{\log_{3/2}^1} = n^0 = 1 \rightarrow T(n) = \Theta(\lg n)$

مثال: (حالت 3)

$T(n) = 3T(n/4) + n \lg n \quad T(1) = c$

$a = 3, b = 4 \quad f(n) = n \lg n \quad n^{\log_b^a} = n^{\log_4^3} = n^{0.793}$

$n \lg n = \Omega(n^{\log_4^{3+\varepsilon}}) \quad af(n/b) \leq cf(n) \quad 3\frac{n}{4}\lg n/4 \leq cn \lg n$

$\rightarrow T(n) = \Theta(n \lg n)$

نکته: $(\lg n)^x < n^y$ به ازای هر $x, y > 0$ برقرار است.

for any integer n, $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n \quad \lceil \lceil n/a \rceil / b \rceil = \lceil n/ab \rceil$

$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$

$\lceil a/b \rceil \leq (a+b-1)/b \quad , \lfloor \frac{a}{b} \rfloor \geq (a-(b-1))/b$

$1 + x \leq e^x < 1 + x + x^2 \quad e^x = 1 + x + \Theta(x^2) \quad \lim_{n\to\infty} (1 + \frac{x}{n})^n = e^x$

فواید استفاده از داده گونه های انتزاعی (ADT)

1- فهم آسان کد 2- اگر نیاز به تغییر پیاده سازی داده گونه های مجرد داشته باشیم

می توان به سادگی و بدون نیاز به تغییر کل برنامه این کار را انجام داد

3- دیگر کد دوم یاد می شود که بتوان کدی که به این شکل نوشته شده را در آینده برای پروژه های دیگر استفاده کرد.

ADT ← List , Graph , Tree , Stack , queue

داده ← PrimitivePaTe Type (بولین، اینتیجر و ...)

1- حافظه را بهتر استفاده می کند 2- متود ندارند.

(مجموعه‌های پویا)

CLRS

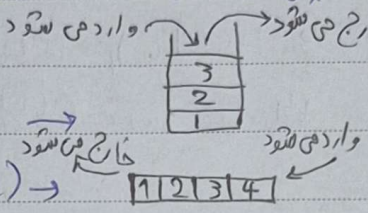
dynamic sets: مجموعه‌هایی که می‌توانند در طول زمان رشد کنند، کوچک شوند یا تغییر کنند

operations: Insert, Delete, Test membership, Minimum, Maximum, Successor(S,x), Predecessor(S,x)

Successor(S,x): یک پرسش که با دریافت یک عنصر x از مجموعه کاملاً مرتب S، اشاره‌گری به عضو بعدی بزرگتر در S را برمی‌گرداند، یا اگر x بزرگترین عضو باشد مقدار NIL را برمی‌گرداند.

Predecessor(S,x): یک پرسش که با دریافت یک عنصر از مجموعه کاملاً مرتب S، اشاره‌گری به عضو قبلی کوچکتر در S را برمی‌گرداند، یا اگر x کوچکترین عضو باشد، مقدار NIL برمی‌گرداند.

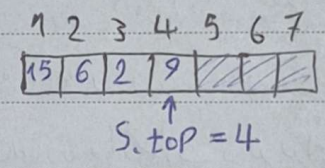
Stack & Queue



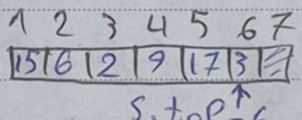
- LIFO: last in first out (stack) → (مثال ظرف شسته)
- FIFO: first in first out (Queue) → (مثال صف نانوایی)

Stack →

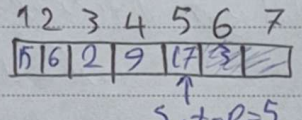
```
STACK-EMPTY(S)
1. if S.top==0
2.    return True
3. else return false
```




```
Push(S,x)
1) S.top = S.top + 1
2) S[S.top] = x
```

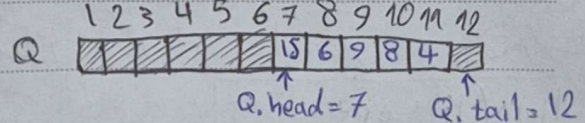



```
Pop(S)
1) if stack-EMPTY(S)
       error "underflow"
   else S.top = S.top - 1
        return S[S.top+1]
```



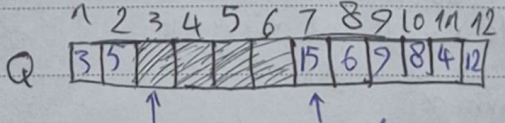
Queue → ENQUEUE(Q,x)

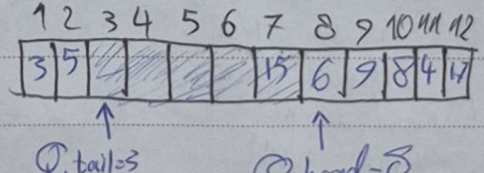
```
1) Q[Q.tail] = x
2) if Q.tail == Q.length
3)    Q.tail = 1
4) else Q.tail = Q.tail + 1
```



DEQUEUE(Q)

```
1) x = Q[Q.head]
2) if Q.head == Q.length
3)    Q.head = 1
4) else Q.head = Q.head + 1
5) return x
```





Subject:
Year. Month. Date. ( )

Exercise

1) نشان دهید چگونه می توان یک صف را با استفاده از دو پشته پیاده سازی کرد. زمان اجرای عملیات های صف را تحلیل کنید

در ابتدا دو stack1 و stack2 را داریم برای EnQueue به stack1 یک عنصر اضافه می کنیم برای DeQueue به صورتی که اگر stack2 خالی بود همه عضرهای stack1 می ریزیم و درون stack2 می ریزیم پس عضر بالای stack2 را برمی گردانیم اگر stack2 خالی نباشد، مستقیم از بالای stack2 حذف می کنیم و برای EnQueue به stack1 اضافه می کنیم O(n)

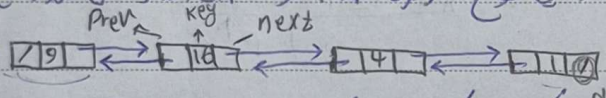
2) نشان دهید چگونه می توان یک پشته را با استفاده از دو صف پیاده سازی کرد. زمان اجرای عملیات های پشته را تحلیل کنید.

برای افزودن (Push) یک عنصر به پشته، عنصر را به Queue1 اضافه می کنیم برای حذف (Pop): تمام عناصر Queue1 به جز آخرین عنصر را به Queue2 منتقل می کنیم. آخرین عنصر را از Queue1 حذف کرده و آن را به عنوان خروجی بازمی گردانیم. نام Queue1، Queue2 را جابجا می کنیم (تا Queue1 همیشه صف اصلی باشد و Queue2 نقش موقت را بازی کند). زمان اجرا Push ← O(1)، زمان اجرا Pop ← O(n)

Linked List:

لیست پیوندی یک ساختار داده است که در آن اشیا به صورت خطی (پشت سر هم) چیده شده اند. برخلاف آرایه ها که ترتیب خطی در آن ها توسط شاخص های آرایه تعیین می شود، در لیست پیوندی ترتیب عناصر توسط اشاره گری در هر شیء مشخص می شود. لیست های پیوندی یک نمایش ساده و انعطاف پذیر برای مجموعه های پویا فراهم می کنند.

انواع: لیست تک پیوندی (singly linked): در این نوع، هر گره تنها به گره بعدی اشاره می کند، یعنی هر گره فقط یک اشاره گر به گره بعدی دارد.



لیست دوپیوندی (Double linked): در این نوع، هر گره به گره قبلی و گره بعدی اشاره می کند. به این ترتیب می توان در لیست هم به سمت جلو و هم به سمت عقب حرکت کرد.

لیست مرتب (Sorted): لیستی که عناصر آن بر اساس ترتیب مشخصی (معمولاً صعودی یا نزولی) مرتب شده اند.

لیست نامرتب (unsorted): لیستی که عناصر آن ترتیب خاصی ندارند.

لیست حلقوی (circular list): در این نوع لیست، گره انتهایی به گره ابتدایی اشاره می کند و به این ترتیب یک حلقه ایجاد می شود.

Linked list operations

```
List-Search(L,k)
  x = L.head
  while x ≠ NIL and x.key ≠ k
      x = x.next
  return x
```

```
List-Insert(L,x)
  x.next = L.head
  if L.head ≠ NIL
      L.head.prev = x
  L.head = x
  x.prev = NIL
```

```
List-Delete(L,x)
  if x.prev ≠ NIL
      x.prev.next = x.next
  else L.head = x.next
  if x.next ≠ NIL
      x.next.prev = x.prev
```

```
List-DELETE'(L,x)
  x.prev.next = x.next
  x.next.prev = x.prev
```

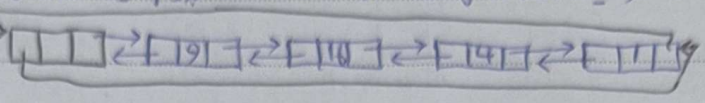
P4PCO

Subject:
Year. Month. Date. ( )

۵۶

Sentinels (نگهبان) L.nil ابتدای لیست (سر لیست) و انتهای لیست (دم لیست) قرار دارد.

L.nil → [| |] L.nil.next به سر لیست اشاره می کند. L.nil.Pre به دم لیست اشاره می کند.



```
List-SEARCH'(L,k)
    x = L.nil.next
5   while x ≠ L.nil and x.key ≠ k
        x = x.next
    return x
List-Insert'(L,x)
    x.next = L.nil.next
    L.nil.next.Prev = x
    L.nil.next = x
    x.Prev = L.nil
10
```

تمرین ۱. آیا می توان عملیات insert را در یک لیست پیوندی یک طرفه در زمان O(1) پیاده سازی کرد؟ حذف (Delete) چطور؟

★ عملیات insert را با O(1) انجام می دهیم:

۱- ایجاد یک گره جدید ۲- تنظیم اشاره گر گره جدید به سر لیست فعلی

۳- به روز رسانی سر لیست به گره جدید

★ عملیات DELETE: برای حذف گره ای با اطلاعات خاص در یک لیست پیوندی یک طرفه، معمولاً نیاز به پیمایش لیست تا یافتن گره مورد نظر داریم. به همین دلیل، این عملیات در بدترین حالت در زمان O(n) انجام می شود. زیرا باید لیست را جستجو کنیم. بنابراین، حذف در زمان O(1) ممکن نیست، مگر اینکه گره مورد نظر به عنوان ورودی به تابع داده شده باشد و نیازی به جستجو آن در لیست نباشد.

Postfix	Prefix	infix
AB+	+AB	(A+B)

Infix to Prefix → A*B + C/D → *AB + /CD → + *AB/CD

(A - B/C) * (A/K - L) → (A - /BC) * (/AK - L) → (- A/BC) * (- /AKL)

→ * - A/BC - /AKL

مثال: چطوری تبدیل Infix to Prefix را با دو تا استک پیاده سازی کنیم؟

دو استک به اسم های operator و operand داریم. در هر مرحله اگر کاراکتری که داریم پیمایش می کنیم عملوند بود آن را در استک operand می ریزیم اگر 'c' (پرانتز باز) بود آن را به operator اضافه می کنیم اگر ')' (پرانتز بسته بود) یک حلقه درون operator می زنیم که تا وقتی به 'c' نرسیدیم دو عنصر از operand برداره و با عنصر بالای operator، Prefix بسازه و توی operand بریزه. اگر کاراکتر عملگر بود /, *, -, + آنگاه بررسی کنیم اگر اولویت آن از عنصر بالای operator بیشتر بود صبرا به استک اضافه کند در غیر اینصورت مثل حالتی که کنترل Prefix می سازیم. حالا که همه عناصر را پیمایش کردیم یک حلقه روی operator می زنیم که تا وقتی خالی نشده همون کاری که کنترل Prefix بسازیم رو انجام بدیم.

دقت کنیم که در این مثال =, +, -, *, / از پرانتز باز اولویت بالاتری دارند.

Subject:
Year. Month. Date. ( )

جزوه

Pattern Matching: الگوریتم های تطبیق الگو تکنیک های محاسباتی هستند که مکان های خاص را پیدا می کنند الگوهای درون یک مجموعه داده، مانند رشته ها یا دنباله ها، و موقعیت آن ها را پیدا کنید. کاربردها: ① جستجو و بازیابی متن (Text Search and Retrieval) ② تحلیل توالی ها (DNA sequence Analysis) ③ داده کاوی (Data Mining) ④ تشخیص گفتار، ⑤ طراحی کامپایلر

الگوریتم ها:
① Naive Pattern Searching algorithm
② KMP algorithm (Knuth-morris-Pratt)
③ RK algorithm (Rabin karp)
④ Z algorithm

① Naive Pattern Searching algorithm (الگوریتم جستجوی الگوی ساده)

سوال: دو تا رشته txt و Patt داریم می خواهیم ببینیم آیا Patt زیر رشته txt است یا نه

```
for i in range(Len(text)-Len(Pattern)+1):
    k=1
    for j in range(Len(Pattern)):
        if Pattern[j] != text[i+j]:
            k=0
            break
    if k==1:
        Print("Yes",i)
```

کدی که خودم زدم

② KMP (Knuth-Morris-Pratt) برای جستجوی الگو در یک متن طراحی شده است که با استفاده از اطلاعاتی که در مورد تطابق های قبلی به دست آمده، از انجام مقایسه های تکراری جلوگیری می کند، به عبارت دیگر، این الگوریتم به جای اینکه از ابتدا مقایسه ها را از نو شروع کند، از جایی که قبلاً مطابقت ها بررسی شده ادامه می دهد.

توضیح مفهوم LPS[] (Longest Prefix suffix): در این الگوریتم، به جای مقایسه مستقیم تمامی کاراکترها در هر مرحله، از یک آرایه به نام LPS[] استفاده می شود که اطلاعاتی در مورد "طول پیشوند صحیحی که هم زمان پسوند هم است" برای هر موقعیت در الگو فراهم می کند.

LPS examples: AAAA → [0,1,2,3] ABCDE → [0,0,0,0,0]

AABAACAABAA → [0,1,0,1,2,0,1,2,3,4,5]

AAACAAAAAC → [0,1,2,0,1,2,3,3,3,4] AAABAAA → [0,1,2,0,1,2,3]

کدش (واقعاً سخته و درکش نیز مرکز بالایی نیاز داره که من ندارم، پس حقوا حقیقتش کن):

```
① lps[0]=0
② len=0
③ i=0
④ if Pat[i]==Pat[len]:
⑤     len+=1
⑥     lps[i]=len
⑦     i+=1
⑧ else:
⑨     if len!=0:
⑩         len=lps[len-1]
⑪     else:
⑫         lps[i]=0
⑬         i+=1
```

PAPCO

Subject:
Year. Month. Date. ( )

۱۱ صل

KMP algorithm: def KMP(Pat, txt):

```
    M = len(Pat)
    N = Len(txt)
    lPS = [0]*M
    i = j = 0
    while i<N:
        if Pat[j] = txt[i]:
            j++, & i++
            if j == M:
                Print("Found" (i-j))
                j = lPS[j-1]
        else:
            if i<N and Pat[j] != txt[i]:
                if j != 0:
                    j = lPS[j-1]
                else:
                    i++
```

الگوریتم Rabin-karp: یک الگوریتم جستجو رشته است که برای پیدا کردن یک الگو در داخل متن به کار می رود. این الگوریتم به جای مقایسه هر کاراکتر متن و الگو به طور مستقیم، از هش (یک نوع کد یا عدد مخصوص) استفاده می کند تا مقایسه ها سریع تر انجام شوند. در این الگوریتم، ابتدا هش الگو و سپس هش زیر رشته های متن محاسبه می شود، سپس هش ها مقایسه می شوند و در صورتی که هش ها برابر بودند، مقایسه دقیق تری روی کاراکترهای متن و الگو انجام می شود تا مطمئن شویم که الگو در متن پیدا شده است.

مراحل الگوریتم به زبان ساده:

۱- محاسبه هش الگو: یک عدد به عنوان هش برای الگو محاسبه می شود.

2- محاسبه هش زیر رشته های متن

3- مقایسه هش ها 4- مقایسه کاراکترها

Rolling Hash: $h(k) = (k[0]b^{L-1} + k[1]b^{L-2} + \dots + k[L-1]b^{0}) \bmod m$

$h(s_{i+1}) = b(h(s_i) - b^{L-1}s[i]) + s[i+L] \bmod m$

Subject:
Year. Month. Date. ( )

Trees : ویژگی های درخت : ① همبند و بدون دور است ② هر دوری بینشان اضافه شود تشکیل دور می دهد ③ همبند است ولی هر یالی حذف شود ناهمبند می شود.

تعریف بازگشتی درخت : 1- اگر T خالی باشد، درختی وجود ندارد.

2- درغیر اینصورت T شامل یک گره به نام ریشه (root) است که از آن حداکثر n درخت دیگر منشعب می شوند. این درخت ها، زیردرخت های T نامیده می شوند و هرکدام از آن ها یک ریشه دارند که توسط یک یال به ریشه T متصل شده اند.

نمایش درخت های ریشه دار: برای نمایش درخت های ریشه دار، می توان از ساختار گره ها استفاده کرد. هر گره شامل اطلاعاتی به نام "کلید (key)" است و همچنین اشاره گرهایی دارد که به گره های دیگر اشاره می کنند. ساختار این اشاره گرها بسته به نوع درخت متفاوت است.

درخت باینری: هر گره حداکثر دو اشاره گر دارد، یکی برای فرزند چپ و دیگری برای فرزند راست.

درخت n-تایی: هر گره حداکثر n اشاره گر دارد که به زیر درخت ها (فرزندان) اشاره می کنند.

ساختار کلی مشابه لیست پیوندی است با این تفاوت که در درخت، هر گره می تواند به بیش از یک گره دیگر اشاره داشته باشد.

عمق گره (Depth): عمق یک گره در درخت باینری به تعداد یال هایی اشاره دارد که باید از ریشه تا آن گره طی شود ← عمق گره نشان می دهد چقدر از ریشه فاصله دارد. برای محاسبه عمق، تعداد یال های بین ریشه و آن گره را می شماریم.

ارتفاع گره (Height)

ارتفاع یک گره در درخت باینری به تعداد یال هایی اشاره دارد که باید از آن گره تا عمیق ترین برگ زیر آن طی شود ← به عبارت دیگر ← ارتفاع گره به معنای تعداد یال هایی است که باید طی شود تا از آن گره به پایین ترین گره در زیر آن برسیم.

ارتفاع درخت: ارتفاع درخت معمولاً به ارتفاع گره ریشه اشاره دارد، زیرا بالاترین نقطه درخت همان ریشه است و ارتفاع درخت توجه به بلندترین مسیر از ریشه تا یک برگ محاسبه می شود.

خلاصه ← عمق از ریشه به سمت پایین تا گره مورد محاسبه می شود / ارتفاع از گره تا عمیق ترین برگ در زیر آن محاسبه می شود.

انواع درخت: براساس تعداد فرزندان: ① Full binary Tree ← هر گره دقیقاً 2 یا فرزند دارد یا هیچ فرزندی ندارد.

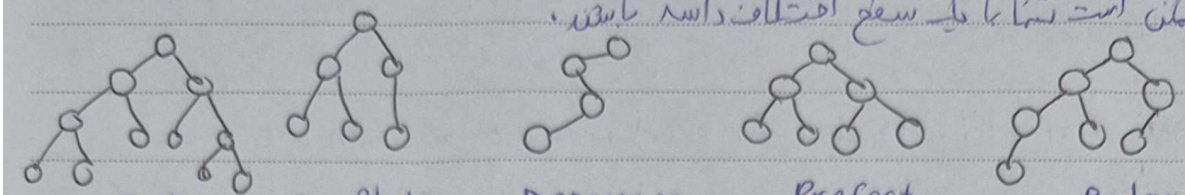
② Degenerated ← در این نوع درخت هر گره تنها یک فرزند دارد، (یک مسیره ایگار)

براساس پر بودن سطح ها: Binary Tree ① Complate BT ← در این درخت، تمام سطح های درخت به طور کامل با گره ها پر شده اند، به جز سطح آخر، در سطح آخر تمام گره ها تا حد ممکن به سمت چپ پر می شوند.

② Prefect BT ← در این نوع درخت، تمام گره های داخلی (گره هایی که برگ نیستند) دقیقاً دو فرزند دارند. تمام گره های برگ در یک عمق مشترک قرار دارند، به این معنا که تمام برگ ها در عمیق ترین سطح درخت قرار گرفته اند.

Subject:
Year. Month. Date. ( )

② Balanced BT ← در درخت باینری متوازن، ارتفاع زیردرخت چپ و راست برای هر گره ممکن است تنها با یک سطح اختلاف داشته باشد.



Full Complate Degenerate Prefect Balanced

For Full BT = Number of leaf nodes = Number of Internal nodes + 1

I = گره‌های داخلی N = I + L ← هم با استدلالی ساده، هم با محاسبه تعداد یال‌ها

L = تعداد برگ N-1 = Edges , 2I = Edges → 2I = I + L - 1 → I + 1 = L

For Perfect BT = Total number of nodes in a with height H is → $2^{H+1} - 1$

واضح!

مثال ①: یک روش بازگشتی با O(n) بگو که با داشتن یک درخت دودویی با n گره، کلید هر گره در درخت را چاپ کند

مثال ②: روش غیر بازگشتی با استفاده از استک با O(n)

① →
```
def dfs(node):
    if node is not None:
        Print(node.key)
        dfs(node.left)
        dfs(node.Right)
```

② →
```
def A(root):
    if root is None: return
    stack = [root]
    while stack:
        node = stack.PoP()
        Print(node.key)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.Left)
```

Huffman coding:
این روش یک تکنیک کدگذاری منبع برای سازی بدون اتلاف داده‌است که از آن برای کاهش تعداد بیت های مورد نیاز جهت نمایش داده‌های متنی استفاده می‌شود.

یک فایل متنی با 100000 کاراکتر در نظر گرفته شده که تنها شامل کاراکتر های 'a' تا 'f' است. فراوانی آن‌ها:

a = 45k c = 12k e = 9k
b = 13k d = 16k f = 5k

اگر به هر کاراکتر یک کد ثابت با طول 3 بیت اختصاص دهیم، برای کل فایل $3 \times 10^5 = 300000$ بیت نیاز داریم. اما با استفاده از کدگذاری متغیر طول (variable-lenght-coding) که در هافمن استفاده می‌شود می‌توان به 224000 بیت کاهش داد

Huffman coding

(a) f:5 e:9 c:12 b:13 d:16 a:45

Huffman coding (c)

(b) c:12 b:13 14 f e d a

(c) f=5 e=9 14 c=12 b=13 25 d=16 a=45

(d) c=12 b:13 14 f:5 e:9 25 d=16 30 a=45

```
n = |C|
Q = C
for i = 1 to n-1
    allocated a new node z
    z.left = x = Extract-Min(Q)
    z.right = y = Extract-Min(Q)
    z.freq = x.freq + y.freq
    insert(Q, z)
return EXTRACT-MIN(Q) // return the root of the tree
```

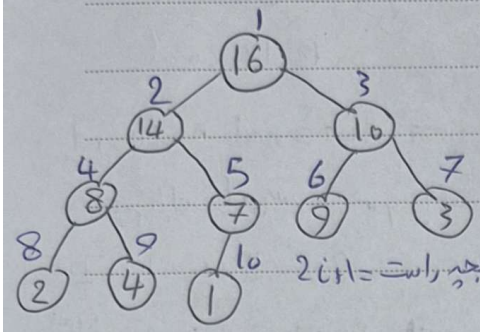
(e) a 55 25 c b 30 14 d f e

(f) 100 a 55 25 c b 30 14 d f e

Subject:
Year. Month. Date. ( )

The (binary) heap is: ① complete binary Tree → درختی که به جز سطح آخر همه سطوح Heap
Types
→ max-heaps ② Has heap Property (ویژگی) به طور کامل با گره‌ها پر شده و در سطح آخر از چپ به راست پر شده.
→ min-heaps

Mas-heap-Property: is that for every node i other than the root:



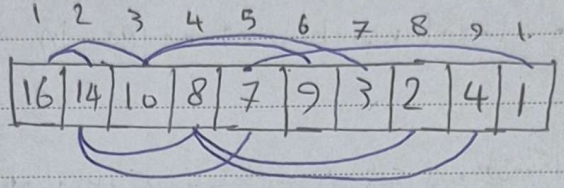
n-i.Parent.key >= n-i.key ✓

PARent(i) → return $\lfloor i/2 \rfloor$

Left(i) → return 2i

Right(i) → return 2i+1

یعنی برای گره i ام ← با اون شبیه گره $\lfloor i/2 \rfloor$ و بچه چپش شبیه گره 2i و بچه راست = 2i+1

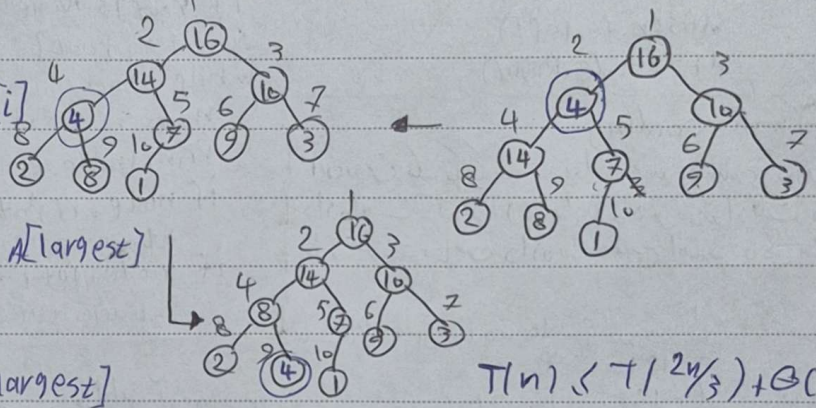


max ، min مقدار تعداد گره‌ها برای هیپ به ارتفاع h ← $2^{h+1}-1$ ، 2^h

MAX-HEAPIFY(A,i)

l = Left(i)
r = Right(i)
if l ≤ A.heap-size and A[l] > A[i]
  largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
  largest = r
if largest ≠ i
  exchange A[i] with A[largest]
  MAX-HEAPIFY(A, largest)

هیپ با n گره ← ارتفاعش: $\lfloor \lg n \rfloor$



$T(n) \leq T(2n/3) + \Theta(1)$

→ $T(n) = O(\lg n)$

Building a heap

A.heap-size = A.length
for i = $\lfloor$ A.length/2 $\rfloor$ downto 1
  MAX-HEAPIFY(A, i)

چون نصف دوم آرایه برگ هستند و کاری بدون تغییر هستند
پس برای هر گره دیگر یکبار MAX-HEAP, را اجرا می‌کنیم.

هر هیپ با n گره در ارتفاع h ام خود حداکثر $\lceil \frac{n}{2^{h+1}} \rceil$ گره دارد.

Running time of building heap

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O(2n) = O(n)$$

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$$

PAPCO

Heap sort(A) $T(n) = O(n \log n)$

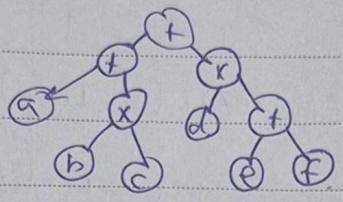
توضیحات خودمونی:

Build-Max-Heap(A)
for i = A.length down to 2
  exchange A[1] with A[i]
  A.heap-size = A.heap-size - 1
  Max-HEAPIFY(A,1)

اول درخت هیپ max درست می‌کنیم بعد در این صورت عضو max ریشه (یا همون عضو اول آرایه خودمون) هست. پس اینو با عضو آخر آرایه swap می‌کنیم و بعدش عضو آخر لیست را در نظر نمی‌گیریم و دوباره درخت می‌سازیم و ...

درخت عبارت = برگ‌هاش عملوند هستند و گره‌های میانی اوپراتور هستند.

a+(b*c)+d*(e+f) →



building expression tree

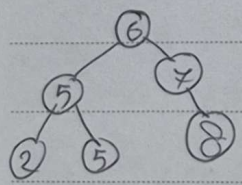
- convert expression to Postfix
- if a character is an operand, add it to the stack.
- if a character is an operator, Pop both values from the stack and make both its children and Push the current nod again

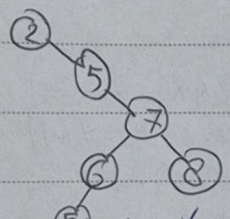
مثال ← $(a+b)*c = ab+cx$

a را به استک اضافه می‌کنیم و یک گره براش درست می‌کنیم، برای b هم همین کار را می‌کنیم برای + هم گره درست می‌کنیم و a را فرزند چپ و b را راست قرار می‌دهیم و به استک اضافه می‌کنیم (a,b پاک شد) c را گره درست می‌کنیم و اضافه می‌کنیم حالا c، ab+ را درمیاریم و ab+ را فرزند چپ X و c را فرزند راست x قرار می‌دهیم و به استک اضافه می‌کنیم

Binary Search tree

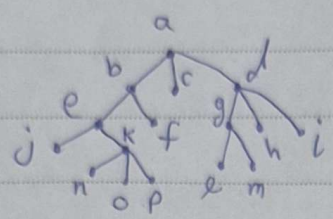
برای هر گره x توی این درخت اگر گره y توی زیردرخت چپ x باشه $y.key \le x.key$ و اگر توی زیردرخت راستش باشه $y.key \ge x.key$





inorder Tree Walk: خاصیت درخت جستجوی دودویی این امکان را به ما می‌دهد که تمام کلیدها را در یک درخت جستجوی دودویی به ترتیب مرتب شده با استفاده از یک الگوریتم بازگشتی ساده، چاپ کنیم.

در inorder ابتدا زیردرخت چپ بعد ریشه بعد زیردرخت راست در pre_order ابتدا ریشه بعد چپ بعد راست و در Post-order اول چپ بعد راست بعد ریشه را پیمایش می‌کنیم.



خلاصه کلاس گذشته نوشته بودم:

Pre-order= a,b,e,j,k,n,o,p,f,c,d,g,e,m,h,i (عینه آدم)

in-order → j,e,n,k,o,p,b,f,a,c,e,g,m,d,h,i (سمت چپ اول میخوره ته چاه)

Post-order → j,n,o,p,k,e,f,b,c,e,m,g,h,i,d,a (تا سمت چپ اول میخوره)

در روش in-order مقادیر را به ترتیب صعودی حاپ می کند:

```
Inorder_Tree_walk(x)
1 if x≠NIL
2     INORDER_TREE_WALK(x.Left)
3     Print x.key
4     INORDER_TREE_WALK(x.right)
```

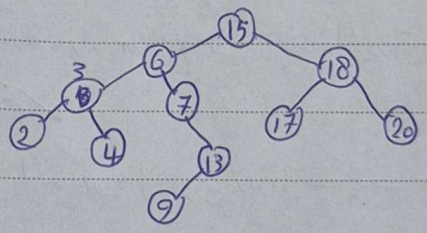
$\Theta(n)$ ✓ زیرا هر گره دقیقاً یکبار بازدید و پردازش می شود.

Querying a binary search tree

به معنای جستجو یک مقدار در درخت جستجو باینری است. اهداف رایج در پرس و جو:

```
TREE-SEARCH(x,k)
1 If x==NIL or k==x.key
2       return x
3 if k<x.key
4     return TREE_SEARCH(x.left,k)
5 else return TREE-SEARCH(x.right,k)
```

مفهوم Iterative_tree-Search: یک روش جستجو برای پیدا کردن مقدار مشخص در درخت است که به جای استفاده از بازگشت (recursion) از یک حلقه تکرار (iteration) استفاده می کند.




```
ITERATIVE_TREE-SEARCH(x,k)
1 while x≠NIL and k≠x.key
2     if k<x.key
3         x=x.left
4     else x=x.right
5 return x
```

```
TREE-Minimum(x)
1 while x.Left≠NIL
2     x=x.Left
3 return x
```

```
TREE-MAXimum(x)
1 while x.right≠Nil
2     x=x.right
3 return x
```

```
Tree-Successor(x)
1 if x.right≠NIL  return Tree-Minimum(x.right)
2 y=x.P // قبلی
3 while y≠NIL and x==y.right
4     x=y , y=y.P
5 return y
```

پسین است یعنی اولین نودی که مقدارش از x بزرگتر است رو نشون میده.

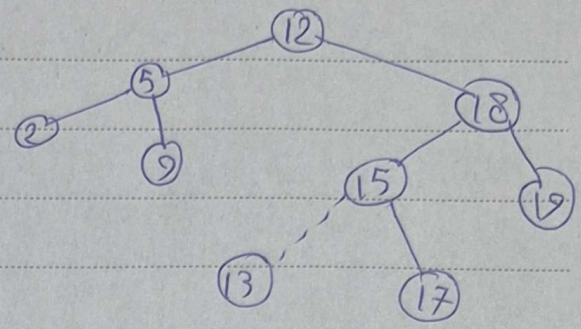
P4PCO
CamScanner

پیاده سازی های Search ، minimum ، maximum ، SUCCESSOR ، PREDECESSOR با O(h) انجام میشود که h ارتفاع درخت است.

Insertion

TREE-INSERT(T,z)

```
1  y=NIL
2  x=T.root
3  while x≠NIL
4      y=x
5      if z.key< x.key
6          x=x.Left
7      else x=x.right
8  z.p=y
9  if y== NIL    T.root=z   //tree T was empty
10 elseif z.key< y.key   y.left=z
11 else y.right = z
```



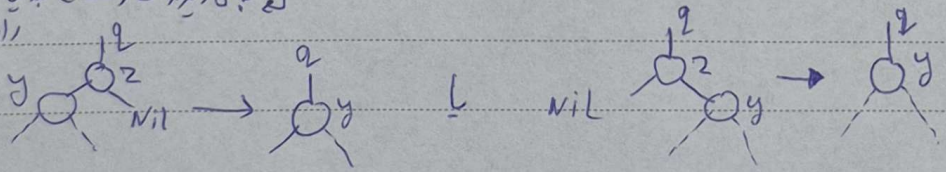
حذف گره از درخت: ③ حالت اساسی وجود دارد:

۱- اگر گره z هیچ فرزندی نداشته باشد (یعنی برگ باشد) به سادگی می توان آن را حذف کرد.
← والد گره z به روزرسانی می شود تا به جای گره z، مقدار NIL را برای فرزندی رفع بگیرد.

2- اگر گره z فقط یک فرزند داشته باشد (چپ یا راست)، فرزند گره z به جای موقعیت z قرار می گیرد.
← والد گره z به روزرسانی می شود تا مستقیماً فرزند z متصل شود. این کار باعث حذف گره z می شود.

3- حذف گره ای که دو فرزند دارد: اگر z دو فرزند داشته باشد، حذف آن پیچیده تر است زیرا باید ساختار BST حفظ شود.
← یافتن جانشین (y): جانشین z کوچکترین مقدار در زیر درخت راست z است (یا بزرگترین مقدار از زیردرخت چپ).
← جایگزینی: گره y را به جای z قرار می دهیم.
← مدیریت زیردرخت ها: زیردرخت های z به گره y متصل می شوند

زیردرخت چپ z تبدیل به زیردرخت چپ y می شود
باقی زیردرخت راست z تبدیل به زیردرخت راست y



حالت ۲: مثل

سوپر توضیح حالت ③: در این حالت گره z یک گره با دو فرزند است و برای حذف آن باید از جانشین z استفاده کنیم. جانشین گره z در درخت جستجو، معمولاً جایگزینی مناسب است که ساختار درخت را حفظ می کند و به ترتیب مرتب سازی درخت آسیب نمی زند.

مراحل انجام این کار: ① پیدا کردن جانشین گره z: جانشین گره z در درخت جستجوی دودویی، گره ای است که به طور معمول در زیر درخت راست گره z قرار دارد و هیچ فرزند چپی ندارد. این گره، کوچکترین گره در زیر درخت راست است ← برای پیدا کردن جانشین، ابتدا به زیر درخت راست گره z می رویم و سپس در این زیر درخت، به دنبال گره ای می گردیم که هیچ فرزند چپی نداشته باشد.

② سه حالت اصلی:

• اگر y فرزند راست z باشد: در این صورت گره z را با گره y جایگزین می کنیم. این کار به این معناست که گره z حذف می شود و گره y به طور مستقیم به جای z قرار می گیرد. در این حالت نیازی به جابجایی فرزند راست y نیست، زیرا y هیچ فرزند چپی ندارد.

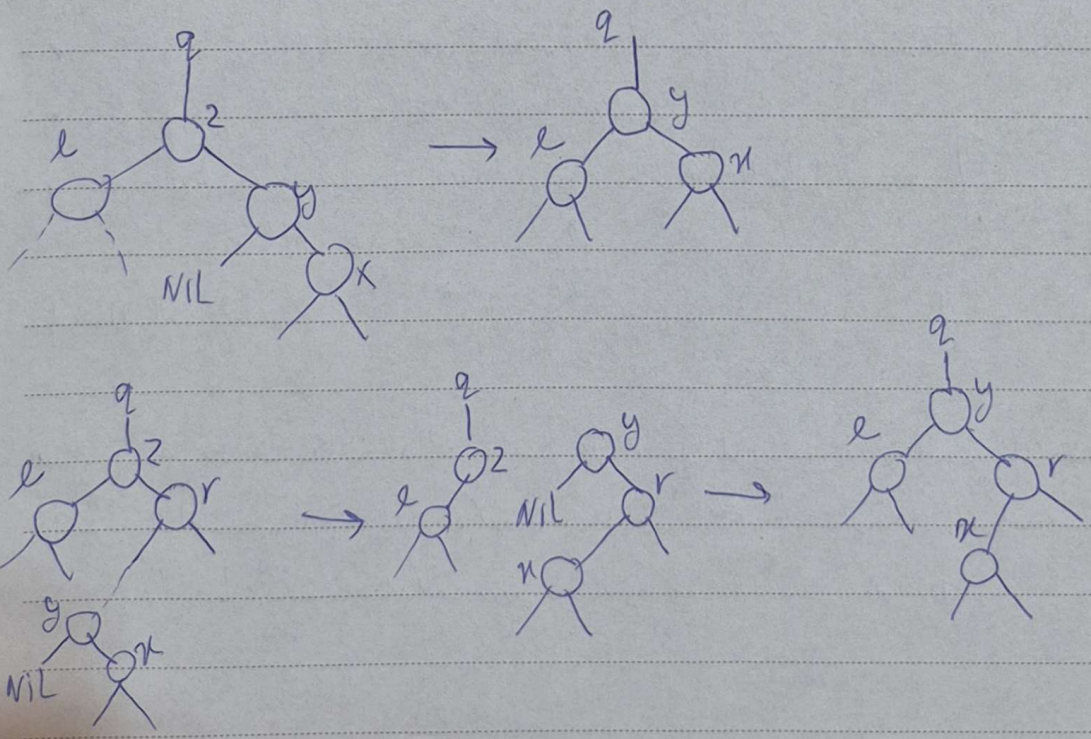
• اگر y از فرزندان راست z دور (نباشد): در این حالت باید گره y را از محل کنونی اش در زیر درخت راست z به جایگاه گره z انتقال دهیم برای این کار ← ابتدا فرزند راست y را به عنوان جانشین گره y معرفی می کنیم سپس گره z را با گره y جایگزین می کنیم.

شرح مراحل: ① پیدا کردن جانشین (successor) گره z:

• به زیر درخت راست z می رویم. • در آنجا به دنبال گره ای می گردیم که هیچ فرزند چپی نداشته باشد (یعنی گره ای که کوچکترین مقدار را در زیر درخت راست دارد). •

② اگر جانشین y فرزند راست z باشد ← • گره z را با y جایگزین می کنیم.

③ اگر جانشین y فرزند راست z نباشد ← گره y را از محل خود در زیر درخت راست z جدا کرده و با فرزند راست y (در صورت وجود) جایگزین می کنیم. سپس گره z را با y جایگزین می کنیم.



transplant(T,u,v) می خواهیم گره v که در درختی دیگر قرار دارد را با گره u که در این گره قرار دارد جایگزین ← ① اگر u ریشه درخت باشد (والدی ندارد):

```
1 if u.p==NIL
2     T.root=v
3 elseif u==u.p.left
4     u.p.left=v
5 else u.p.Right=v
6 if v≠NIL
7     v.p=u.p
```

② ← ریشه درخت T.root به v تغییر داده می شود.
③ اگر u فرزند چپ والدش باشد: ∘ فرزند چپ والد u به v تغییر می کند
④ اگر u فرزند راست والدش باشد: ∘ فرزند راست والد u به v تغییر می کند
⑤ اگر u ≠ NIL باشد: ∘ والد v برابر با والد u تنظیم می شود.

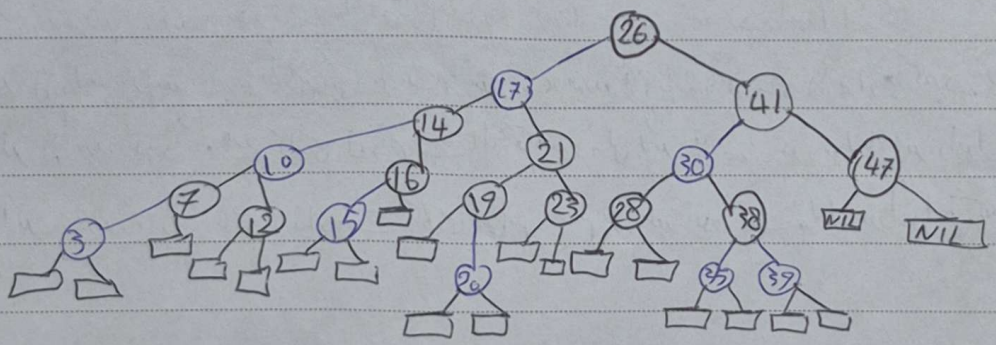
Deletion

```
1  if z.left==NIL
2      Transplant(T,z,z.right)
3  else if z.right==NIL
4      Transplant(T,z,z.left)
5  else y=TREE-minimum(z.right)
       if y.p≠z
           TRANSplant(T,y,y.right)
           y.right=z.right
           y.right.p=y
       Transplant(T,z,y)
       y.left=z.left
       y.left.p=y
```

نکته: ارتفاع مورد انتظار یک درخت جستجوی دودویی که به صورت تصادفی با n کلید متمایز ساخته شده، از مرتبه O(log n) است

درخت‌های Red-black ویژگی‌ها

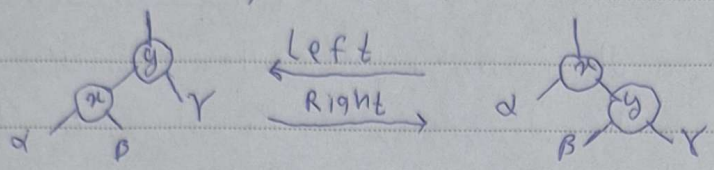
① هر گره ∘ یا قرمز است یا سیاه:
② ریشه سیاه است: ③ هر برگ (NIL) سیاه است.
④ اگر یک گره قرمز باشد، هر دو فرزند آن سیاه هستند.
⑤ برای هر گره، همه مسیرهای ساده از آن گره تا برگ‌های فرزندش، تعداد یکسانی از گره‌های سیاه دارند



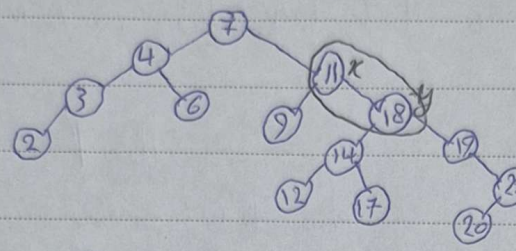
black-height (ارتفاع سیاه) یک گره: ارتفاع سیاه یک گره (که با bh(x) نمایش داده می شود) برابر است با تعداد گره های سیاه در هر مسیر ساده از گره x تا یک برگ، به جز خود x.
∘ این ارتفاع شامل متغیر گره های سیاه است.

A red_black tree with n internal nodes has height at most $2\lg(n+1)$. :لم

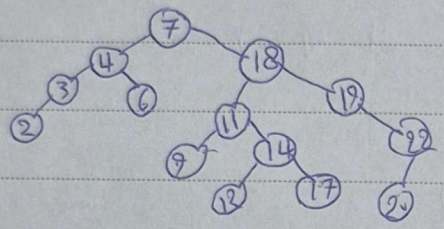
Left-Rotate(T,x) and RIGHT-ROTATE(T,y)



Example:



→



```
Left-Rotate(T,x)
1) y=x.right
2) x.right=y.left
3) if y.left ≠ T.nil
4    y.left.p=x
5  y.p=x.p
6  if x.p==T.nil  T.root=y
7  eslif x=x.p.left x.p.left=y
8  else x.p.right=y
9  y.left=x
10 x.p=y
```

Insertion

ابتدا محل قرارگیری راس جدید را بر اساس اینکه درخت BST است پیدا کنید و یک گره با رنگ قرمز درج کنید.

• اگر پدر گره جدید سیاه است: نیازی به انجام کاری نیست.

اگر پدر گره جدید قرمز است: پدر و عمو را سیاه کنید و جد را قرمز کنید و همین مراحل را برای جد تکرار کنید.

اگر عموی گره جدید سیاه است: ابتدا با چرخش مناسب حول پدر، گره جدید و پدر و جد را روی یک راستا قرار دهید، رنگ پدر و جد را عوض کنید و جد را بسمت عمو بچرخانید.

Deletion

اگر y راسی باشد که به طور فیزیکی حذف می شود و x فرزند آن باشد، x جای y قرار می گیرد. فرض کنید P پدر و S برادر x باشد.

اگر y قرمز بود که هیچ کاری لازم نیست انجام شود ولی اگر y سیاه بود مراحل زیر را برای اصلاح درخت انجام می دهیم:

• اگر x قرمز یا ریشه باشد، نیازی است کاری انجام دهیم در غیر اینصورت مراحل زیر را تکرار می کنیم:

Year: Month: Date: Subject:

Sorting

حفظ ترتیب نسبی عناصر:

• الگوریتم های متعادل (stable): ترتیب نسبی عناصر با کلید های یکسان قبل و بعد از اجرای الگوریتم مرتب سازی یکی است.

• الگوریتم های نا متعادل (unstable): ترتیب نسبی عناصر با کلید های یکسان قبل و بعد از اجرای الگوریتم مرتب سازی لزوماً یکی نیست.

موقعیت داده ها در زمان مرتب کردن:

الگوریتم های مرتب ساز داخلی: همه داده ها هنگام مرتب شدن در حافظه هستند و دسترسی به آنها سریع است.

الگوریتم های مرتب ساز خارجی: همه داده ها هنگام مرتب شدن در حافظه نیستند و دسترسی به آنها کند است.

پس تعداد دسترسی به عناصر خیلی اهمیت دارد.

★ پیچیدگی الگوریتم های مرتب سازی غیر مبتنی بر مقایسه $\Omega(n)$ هستند.

★ پیچیدگی الگوریتم های مرتب سازی مبتنی بر مقایسه $\Omega(n \log n)$ هستند.

چون درخت تصمیم $n!$ برگ دارد، پس ارتفاع آن حداقل $\log n!$ است پس $n \log n$ حداقل زمان است.

Count sort: همونی که اول تعداد هر عدد رای شمار یعنی روش مارشال سام می زنیم و بعد از آخر مرتب می کنیم و یکی از آرایه count کم می کنیم.

کدی که براش زدم:

```
for(int i=0; i<n; i++){
    cin >> a[i];
    b[a[i]]++;
    mx = max(mx, a[i]);
}
for(int i=1; i<=mx; i++) : b[i] += b[i-1];
for(int i=n-1; i>=0; i--) : c[--b[a[i]]] = a[i];
for(int i=0; i<n; i++) : cout << c[i] << " ";
```

پیچیدگی الگوریتم: $T(n) = \Theta(k) + \Theta(n) = \Theta(k+n) \rightarrow O(n)$

Radix Sort ← به نظام حفظ ترین الگوریتم است.

برای مرتب کردن n عدد که رقمی استفاده می شود. اعداد بر حسب کم ارزش ترین رقم با یک الگوریتم متعادل مرتب می شوند و پس بر اساس دومین کم ارزش ترین عدد مرتب می شوند، به همین ترتیب تا با ارزش ترین رقم.

کدی کدرم واسش:

```
void counting_sort(arr, n, exp):
    for i(0 to n): b[(arr[i]/exp)%10]++
    for i(1 to 10): b[i] += b[i-1];
    for i (n-1 to 0): c[--b[(arr[i]/exp)%10]] = arr[i];
    for i (0 to n): arr[i] = c[i]

void radix_sort(arr, n):
    mx = *max_element(arr, arr+n);
    for(exp=1; mx/exp > 0; exp *= 10) : counting_sort(arr, n, exp);
```

$K \in O(n) = O(n)$

دفاتر مرجان MARJAN

CamScanner

Bucket Sort: خیلی چیز خوبیه: همه عناصر ها بین ۰ تا ۱ هستن که یکه به k دسته از $(0\text{ تا }\frac{1}{k})$، $(\frac{1}{k}, \frac{2}{k})$ و ... $(\frac{k-1}{k}, 1)$ تقسیم می‌کنه و اعداد توهر دسته را با روش های دیگه مرتب می‌کنه بعد کنار هم می‌ذاره که O های مختلف داره.

اینا الگوریتم های غیرمبتنی بر مقایسه است.

حالا الگوریتم های مبتنی بر مقایسه:

① Insertion Sort (درجی)
② Selection Sort (انتخابی)
③ Bubble Sort (حبابی)
④ Merge Sort (ادغامی)
⑤ Quick Sort (سریع)
⑥ Heap Sort (هرمی)

① Insertion: همون ساده ترینه است که عناصر یکی یکی به سمت مرتب شده اضافه می‌شوند و در محل مناسب قرار می‌گیرند.

② Selection: هر بار از این مرحله، از این کوچکترین عنصر انتخاب می‌شود و با عنصر بعد از محدوده مرتب شده تعویض می‌شود.

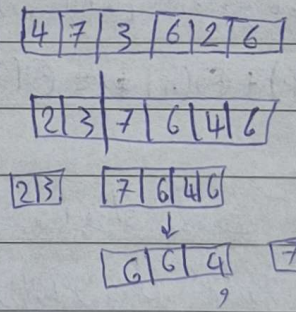
③ Bubble: در هر بار (از ۱ تا n-۱ بار)، هر دو عنصر مجاور با هم مقایسه می‌شوند و احتمالاً تعویض می‌شوند. (همیشه عنصرهای مجاور مقایسه می‌شوند)

④ Merge: یعنی چی؟ خداشی: هر مرحله نصف می‌کنی و مرتب می‌کنی و مرج می‌کنی مرج هم دوتا یو یتر می‌کنیم هر کدوم کوچکتر بود رو میریزی جلو.

⑤ Quick: یک عنصر را به عنوان محور (Pivot) انتخاب کن ← عناصر را به دو جزء تقسیم کن: عناصر کوچکتر از محور و عناصر بزرگتر یا مساوی محور ← الگوریتم را به صورت بازگشتی تکرار کن:

```
Quick Sort(A,P,r)
  if P<r then
    q = Partition(A,P,r)
    Quick Sort(A,P,q)
    Quick Sort(A,q+1,r)
```

Pivot = 4



23 ص
اسلایدی به صورت خیلی خوبه

```
Partition(A,P,r)
  x ← A[P]
  i ← P-1
  j = r+1
  while (1):
    j--;
    until A[j] ≤ x
    i++;
    until A[i] ≥ x
    if i<j then swap(A[i],A[j])
    else return j
```

اگر محور انتخاب شده، بزرگترین کلید یا کوچکترین کلید باشد، بدترین حالت اجرای الگوریتم اتفاق می‌افتد.

دفاتر مرجان MARJAN

CamScanner

Year: Month: Date: Subject:

Heap sort الگوریتم: - با عنصر یک max heap بساز

- n بار بزرگترین عنصر heap را حذف کن

مرحله اول: - ساختن max heap

آرایه اولیه →

1	2	3	4	5	6	7	8	9	10
26	5	77	1	61	11	59	15	48	19

واسه دین که سرچ کن

بعد از ساختن max heap →

1	2	3	4	5	6	7	8	9	10
77	61	59	48	19	11	26	15	1	5

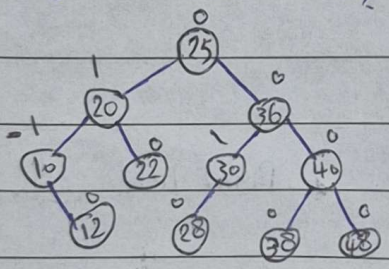
مقایسه الگوریتم های مرتب سازی

	بهترین حالت	بدترین حالت	حالت متوسط	Stable
insertion sort	$o(n)$	$o(n^2)$	$o(n^2)$	✓
Selection sort	$o(n^2)$	$o(n^2)$	$o(n^2)$	✗
Bubble sort	$o(n^2)$	$o(n^2)$	$o(n^2)$	✓
Merge sort	~~o(n)~~ $o(n\lg n)$	$O(n\lg n)$	$O(n\lg n)$	✓
Quick sort	$o(n\lg n)$	$o(n^2)$	$O(n\lg n)$	✗
Heapsort	$o(n\lg n)$	$o(n\lg n)$	$O(n\lg n)$	✗

Avl tree

AVL نوع خاصی از درخت جستجو دودویی است - که همیشه متعادل (Balanced) باقی می ماند برای هر گره x در درخت ارتفاع زیر درخت سمت چپ و زیر درخت سمت راست x می تواند متفاوت باشد، اما این تفاوت نباید بیش از 1 باشد.

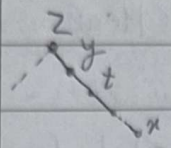
این اعداد روی گره ها تفاوت زیر درخت چپ از زیر درخت راست است.



insert مراحل درج در درخت AVL:

۱- گره x را مانند یک درخت جستجوی دودویی معمولی درج می کنیم. یعنی مقادیر کوچکتر به زیر درخت چپ و مقادیر بزرگتر به زیر درخت راست می روند. / پس از درج x از گره x به سمت بالا (به سمت ریشه) حرکت می کنیم، Balance factor (ضریب تعادل) را در هر گره بررسی می کنیم. / اولین گره ای که مقدار BF آن از محدوده $[-1, 0, 1]$ خارج شده است (یعنی $BF = \pm 2$)، به عنوان گره نامتعادل (z) انتخاب می شود.

Year: Month: Date: Subject:

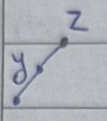


یافتن گره های z و y و t : z = اولین گره ای که نامتعادل است.

y = فرزند z در مسیری که از x به z رسیدیم.

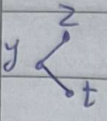
t = فرزند y در مسیری که از x به z رسیدیم.

4 حالت ممکن برای عدم تعادل :



1) Left-Left (چپ چپ) y فرزند چپ z و t فرزند چپ y است.

مشکل : زیردرخت چپ بیش از حد بلند شده / راه حل: چرخش ساده به راست روی z



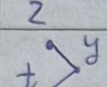
2) Left-Right (چپ راست) : y فرزند چپ z و t فرزند راست y است.

راه حل: چرخش ساده به چپ روی y / سپس چرخش ساده به راست روی y



3) Right Right (راست راست) : y فرزند راست z و t فرزند راست y است.

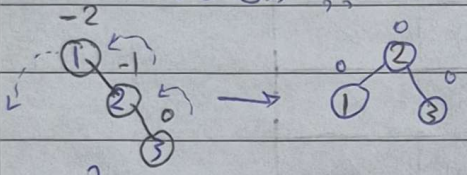
راه حل: چرخش ساده به چپ روی z.



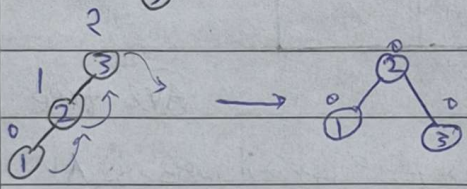
4) Right-Left (راست چپ) : y فرزند راست z و t فرزند چپ y است.

راه حل: چرخش ساده به راست روی y / سپس چرخش ساده به چپ روی z.

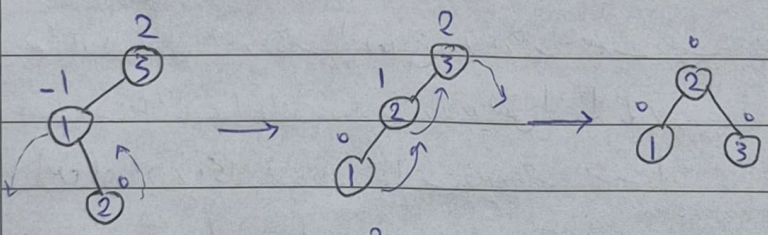
مثال ← Left left



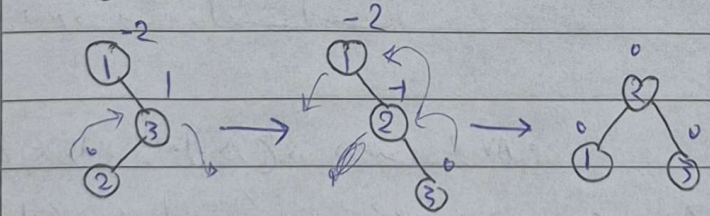
مثال ← Right Right



مثال ← Left right



مثال ← Right Left



Avl tree height

$N(h) = 1 + N(h-1) + N(h-2)$ ، مینیمم برگ های زیردرخت راست $N(h-2)$ ، مینیمم برگ های زیردرخت چپ $N(h-1)$

$N(h-1) > N(h-2) \rightarrow N(h) > 2N(h-2)$ بازگشتی حل کنیم $\rightarrow N(h) > 2^{h/2}$

$\rightarrow h < 2 \log N_h$

Year: Month: Date: Subject:

B-Tree یک درخت B، یک درخت ریشه دار است (که ریشه آن T.root است) با ویژگی های زیر:

1- هر گره x دارای ویژگی های زیر است: x.n تعداد کلید هایی که در گره x ذخیره شده اند. a→

کلید های x.key1, ..., x.keyn به صورت سورت شده است. b→

x.leaf یک مقدار بولی که مقدار آن True است اگر x یک گره برگ باشد، و False اگر گره داخلی باشد. c→

2- هر گره x داخلی x.n+1 پوینتر به فرزندان خود است

سوال: مگه هر گره n بچه نداره چرا من n+1 پوینتر به بچه ها و میدی؟

هر یک از این کلید ها یک بازه را جدا می کنند به عنوان مثال، اگر n کلید داشته باشیم، بازه ها به این شکل خواهند بود: قبل از x.key1، بین x.key1 و x.key2، ... .

3- کلید های x.keyi بازه های کلید های ذخیره شده در هر زیردرخت را جدا می کنند:

اگر ki کلیدی باشد که در زیردرخت با ریشه x.ci ذخیره شده است آنگاه:

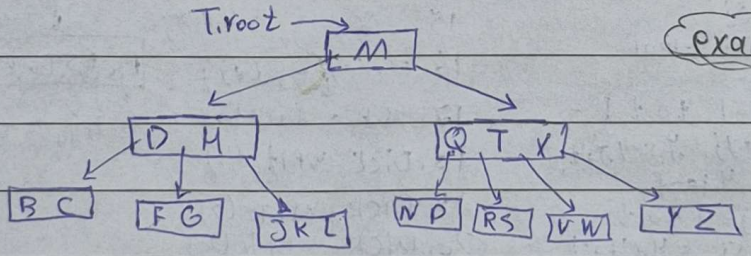
$k_1 \le x.key_1 \le k_2 \le x.key_2 \le \cdots \le x.key_{x.n} \le k_{x.n+1}$

4- همه گره های برگ، عمق یکسانی دارند که همان ارتفاع درخت (h) است.

5- درخت B دارای محدودیت هایی روی تعداد کلید های هر گره است که به صورت زیر تعریف می شود: 1- محدوده حداقل تعداد کلیدها: همه راس های به جز ریشه حداقل t-1 کلید دارند. که t یک عدد ثابت و $t \ge 2$ است که به آن درجه حداقل درخت B گفته می شود. اگر گره ای غیر از ریشه باشد، حداقل t فرزند خواهد داشت، گره ریشه هم حداقل 1 کلید دارد.

2- محدوده حداکثر تعداد کلیدها: هر گره می تواند حداکثر (2t-1) کلید داشته باشد به همین ترتیب هر گره می تواند حداکثر 2t فرزند داشته باشد. اگر یک گره دارای 2t-1 کلید باشد به آن گره پر (Full Node) گفته می شود.

example



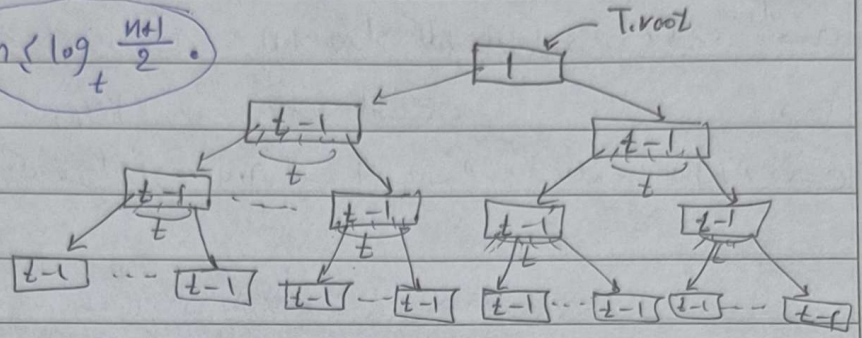
* توجه کن پوینتر ها طبق ترتیب بازه هاست مثلا آبی DH یکی قبل D هست یکی بین D و H یکی بعد H.

Year: Month: Date: Subject:

the height of a B-tree

if $n \geq 1$, then for any n-key B-tree T of height h and minimum degree $t \geq 2$, $h \leq \log_t \frac{n+1}{2}$.

depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$



Searching a B-tree

```
B-Tree-Search(x, k)
  i=1
  while i≤x.n and k>x.key_i
      i = i+1
  if i≤x.n and k==x.key_i
      return(x,i)
  elseif x.leaf
      return NIL
  else Disk-Read(x.c_i)
      return B-TREE-SEARCH(x.c_i, k)
```

الان قدری کد کنده هستیم و از جدید ترین پدر شروع می کنیم و جلو می رویم تا وقتی که $k < x.key_i$ باشد. اگر مساوی بود که همین، اگر بزرگ بود هم نیل اگر باز کمتر نبود بچه x می رویم.

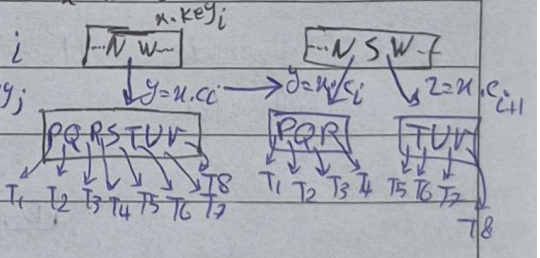
Creating an empty B-tree

```
B-Tree_Create(T)
  x = Allocate_node()
  x.leaf = true
  x.n = 0
  Disk-WRITE(x)
  T.root = x
```

B-Tree_create requires O(1) disk operation and O(1) CPU time.

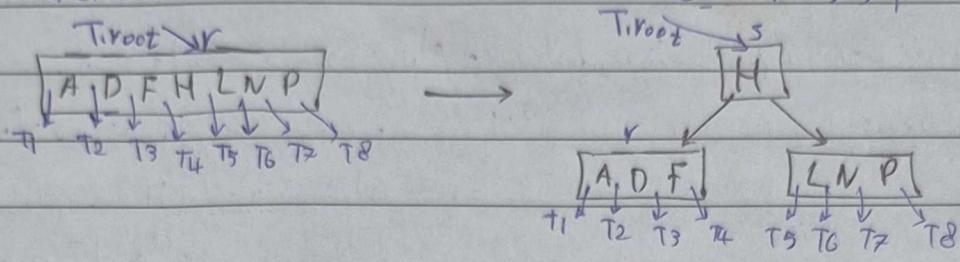
insert

```
1.  z = Allocate_node()
2.  y = x.c_i
3.  z.leaf = y.leaf
4.  z.n = t-1
5.  for j=1 to t-1
6.      z.key_j = y.key_{j+t}
7.  If not y.leaf
8.      for j=1 to t
9.          z.c_j = y.c_{j+t}
10. y.n = t-1
11. for j = x.n+1 downto i+1
12.     x.c_{j+1} = x.c_j
13. x.c_{i+1} = z
14. for j = x.n downto i
15.     x.key_{j+1} = x.key_j
16. x.key_i = y.key_t
17. x.n = x.n+1
18. Disk-WRITE(y)
19. Disk-WRITE(z)
20. Disk-WRITE(x)
```

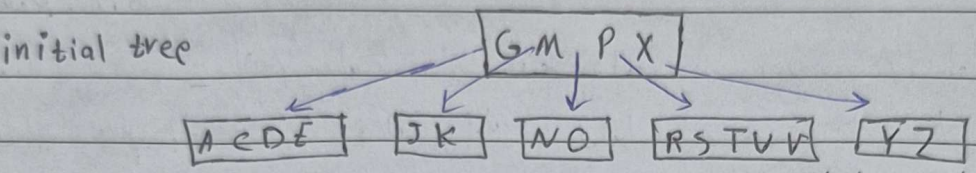


Year: Month: Date: Subject:

insert:



گره ها رو نوشتم تو خود Pdf ببین



ادامش زیاده ولی آسونه خودت رو ببین.

delete

① اگر Key K که می خواهیم حذف کنیم برگ باشد و حداقل t تا باشد عین آدم یکی شو حذف می کنیم

② اگر از نود های داخلی می خواستیم حذف کنیم اگه بچه چپ حداقل t تا داشت بزرگترین اولاد میاری جای K ، اگه بچه راست حداقل t تا داشت کوچکترین اولاد میاری جاش اگه هردو کمتر از t بود مرج می کنیم صرفا چون حجمش از 2t-1 کمتر می شه.

Hash tables

جدول درهم سازی (Hash table) یک ساختار داده موثر برای پیاده سازی دیکشنری ها است. با فرضیات سفارش، میانگین زمان جستجوی یک عنصر در جدول درهم سازی O(1) است. استفاده از Hash Table برای کاربردهایی که به عملیات search, insert, delete محدود هستند، یکی از کارآمدترین راه حل ها است.

تاکنون با دو ساختار مهم برای نگهداری از مجموعه ای از داده ها، آشنا شده ایم = آرایه و لیست پیوندی حال هش تیبل یکی از ساختار داده ای است که سرعتش خیلی بیتر از این دو تاس و O(1) عملیات را انجام میده.

تابع هش چیست؟ تابع هش، یک الگوریتم محاسباتی است که یک داده را به عنوان ورودی میگیرد و یک عدد (کد هش) را به عنوان خروجی برمی گرداند. تابع هش باید این دو ویژگی را دارا باشد

- به ازای ورودی یکسان، خروجی یکسان برگرداند
- ورودی های متفاوت را، به اعداد متفاوتی تبدیل کند.

Year: Month: Date: Subject:

آرایه هش چیست؟ آرایه ای است که در آن جای هش جدول در آن ذخیره میشود، اندازه این آرایه ثابت است و اینکه هر داده در کدام خانه قرار بگیرد توسط تابع هش تعیین می شود. یعنی ابتدا داده را به تابع هش میدهیم و یک عدد از آن میگیریم، آن عدد نشان میدهد که آن داده در کدام خانه از آرایه ذخیره خواهد شد.

مثال هش:

مثلاً فرض کن می خواهیم یک لیست مدیریت از محصولات فروشگاه درست کنیم. اول یک آرایه خالی داریم با اندازه بزرگ. حالا نام اولین محصول را به تابع هش می دهیم و از تابع یک عدد خروجی می گیریم. مثلاً "کالباس سوئیسی" می دهیم و تابع هش 274 میده. ما اطلاعات محصول کالباس سوئیسی را در خانه 274 از آرایه ذخیره میکنیم همینکار را برای محصولات دیگر میکنیم. جستجو محصولات در لیست هم بسیار سریع خواهد بود، کافیست نام محصول را به تابع هش بدهیم و محل عددی که محل ذخیره سازی اطلاعات محصول را نشان میدهد.

Hash function

① روش تقسیم ← $h(k) = k \bmod m$ ⟵ معمولاً m را عددی اول میگیریم تا برخورد ها حداقل شوند.

② روش ضرب ← $h(k) = \lfloor m(kA \bmod 1) \rfloor$

"$kA \bmod 1$" means the fractional part of kA, that is, $kA - \lfloor kA \rfloor$

Open Addressing (آدرس دهی باز)

این تکنیک برای مدیریت برخورد ها (collisions) در جدول درهم سازی استفاده می شود. به جای استفاده از ساختارهای داده ای اضافی (مانند لیست پیوندی) در جدول خود مکان دیگری برای کلید جدید پیدا می کنیم

```
Hash-Insert(T,k)
i = 0
repeat
    j = h(k,i)
    if T[j] == NIL
        T[j] = k
        return j
    else i = i+1
until i == m
error "hash table overflow"
```

```
Hash-search(T,k)
i = 0
repeat
    j = h(k,i)
    if T[j] == k
        return j
    i = i+1
until T[j] == NIL or i == m
return NIL
```

Year: Month: Date: Subject:

(جستجو خطی) Linear Probing

۱. تعریف: جستجو خطی یک روش برای مدیریت برخورد (collisions) در جدول درهم سازی است. اگر مکان اولیه تعیین شده توسط تابع هش ($h'(k)$) اشغال شده باشد، از توالی خطی برای پیدا کردن مکان خالی استفاده می‌کنیم.

$$h(k,i) = (h'(k) + i) \bmod m$$

$h'(k)$: تابع هش اولیه / i = تعداد دفعات تلاش برای پیدا کردن مکان مناسب / m = اندازه جدول

مشکل Primary cluster (یا clustering): زمانی رخ می‌دهد که گروهی از اسلات‌های پشت سر هم اشغال شده باشند. احتمال پر شدن اولین اسلات خالی پس از این گروه بیشتر است. این مشکل باعث طولانی‌تر شدن زنجیره‌ها و افزایش زمان جستجو در حالت میانگین می‌شود.

Quadratic Probing (جستجو درجه دوم)

تعریف: این روش برای مدیریت برخوردها از یک تابع هش درجه دوم استفاده می‌کند تا مکان‌های جدید برای ذخیره یا جستجوی کلیدها پیدا کند. فرمول $\leftarrow$

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m.$$

مشکل Secondary clustering: در این روش، دنباله مکان‌های بررسی شده کاملاً به مقدار اولیه تابع هش $h'(k)$ وابسته است. در حالت جستجو خطی، توالی‌های بررسی برای کلیدهای مختلف با $h'(k)$ متفاوت هستند، که مشکل تجمع اولیه را کاهش می‌دهد. اما همچنان ممکن است به دلیل توالی محدود، برخورد رخ دهد.

مقایسه:

ویژگی	Linear Probing	Quadratic Probing
نحوه جستجو	بررسی خطی مکان‌های بعدی	بررسی مکان‌ها بر اساس یک توالی درجه 2
Primary clustering	وجود دارد	کاهش می‌یابد
Secondary clustering	کاهش نمی‌یابد	کاهش می‌یابد
پیچیدگی پیاده‌سازی	ساده‌تر	پیچیده‌تر

Double Hashing (هشینگ دوگانه):

تعریف: هشینگ دوگانه یک روش برای مدیریت برخوردها در جدول است که از دو تابع هش مستقل برای تولید توالی مکان‌های بررسی استفاده می‌کند. فرمول:

$$h(k,i) = (h_1(k) + i h_2(k)) \bmod m$$

$$h_1(k) = k \bmod m, \quad h_2(k) = 1 + (k \bmod m')$$

Year: Month: Date: Subject:

در فرمول های قبل:

$h(k,i) = (h_1(k) + i h_2(k)) \bmod m$

$h_1(k) = k \bmod m \quad h_2(k) = 1 + (k \bmod m) = 1 + (k \bmod (m-1))$

$h_1(k)$ تابع هش اولیه، $h_2(k)$ تابع هش دوم که برای جایگزینی استفاده می شود.

i = تعداد دفعات تلاش برای پیدا کردن مکان خالی. m : اندازه جدول

شرط مهم: مقدار $h_2(k)$ باید نسبت به m (اندازه جدول) نسبتاً اول باشد تا تقسیم شود. همه خانه های جدول در صورت نیاز بررسی شوند. برای مثال: $h_2(k) = 1 + (k \bmod (m-1))$

مزیت: برخلاف جستجو خطی یا درجه 2، توالی خانه ها به دو تابع هش مستقل بستگی دارد و احتمال تجمع اولیه و ثانویه کاهش می یابد.

تمرین: کلید های 88, 17, 28, 15, 4, 31, 22, 10 را باید در جدولی با طول $m=11$ قرار دهیم. از روش های مختلف مدیریت برخورد ها استفاده کنید؟

Linear Probing = $h(k,i) = (h'(k) + i) \bmod m$

$10 \xrightarrow{\bmod 11} 10$ ✓ $\quad 22 \xrightarrow{\bmod 11} 0$ ✓ $\quad 31 \xrightarrow{\bmod 11} 9$ ✓ $\quad 4 \xrightarrow{\bmod 11} 4$ ✓

$15 \xrightarrow{\bmod 11} 4 \xrightarrow{\text{خانه بعدی}} 5$ ✓ $\quad 28 \xrightarrow{11} 6$ ✓ $\quad 17 \xrightarrow{11} 6 \xrightarrow{\text{خانه بعدی}} 7$ ✓

$88 \xrightarrow{11} 0 \xrightarrow{\text{خانه بعدی}} 1$ ✓ $\quad 59 \xrightarrow{11} 4 \xrightarrow{\text{خانه های بعدی}} 8$ ✓

Quadratic Probing = $h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \quad c_1=1, c_2=3$

طبق حل دیگر فقط 15, 17, 88, 59 را بررسی می کنیم

$15 \to (4+1+3) \bmod 11 = 8$ ✓

$17 \to (6+1+3) \bmod 11 = 10 \to (6+2+12) \bmod 11 = 9 \to (6+3+27) = 3$ ✓

($i=1$ ، $i=2$ ، $i=3$)

$88 \to (0+1+3) \bmod 11 = 4 \to (0+2+12) \bmod 11 = 3 \to (0+3+27) = 8$

$\to (0+4+48) = 8 \to (0+5+75) = 3 \to (0+6+108) = 4 \to (0+7+147) = 0$

$\to (0+8+192) = 2$ ✓

Double hashing: -3

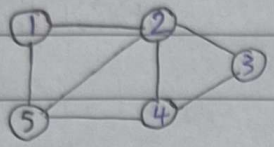
$15 \to (4 + (1,5)) = 10 \times \to (4 + 2(1+5)) = 5$ ✓ وضعیت آخر ✓

$E[X] = \binom{n}{2} \cdot \frac{1}{n^2} = \frac{n^2-n}{2} \times \frac{1}{n^2} < \frac{1}{2}$ ← امید ریاضی برخورد در هش

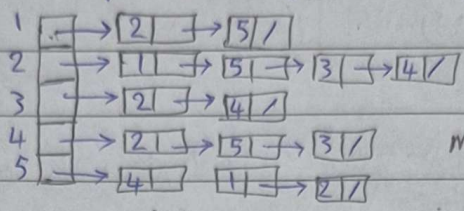
$m = n^2$ آخر

Year: Month: Date: Subject:

گراف : با دو روش میتوانیم گراف ها را ذخیره کنیم ۹ : اگر گراف پراکنده باشد یعنی تعداد یال ها خیلی کمتر از تعداد راس های بتوان ۲ باشد آنگاه از لیست مجاورت استفاده می کنیم.
ولی اگر متراکم یا کامل بود بهتر است از ماتریس استفاده کنیم :



(a)



(b)

memory : $\Theta(V+E)$

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

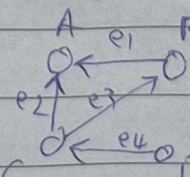
(c)

memory : $\Theta(|V|^2)$

گراف جهت دار و وزن دار :
با همین دو راه ذخیره می شوند و فرقشون هم واضح دیگه ، یال ها را ابتدا دو طرفه میکشیم.
گراف وزن دار و نحوه ذخیره وزن آن :
در یک گراف وزن دار ، هر یال (u,v) دارای یک مقدار عددی (وزن) است که می تواند نشان دهنده هزینه ، فاصله ، زمان ، یا سایر معیارهای مشابه باشد. تابع وزن w وظیفه اختصاص این مقادیر به یال ها را بر عهده دارد.
نمایش با لیست مجاورت :
در این روش ، هر راس لیستی از راس هایی که به آن ها متصل است دارد. اگر یال ها وزن داشته باشند ، وزن هر یال نیز به همراه راس مقصد ذخیره می شود.
مثال: اگر یال (u,v) با وزن 5 وجود داشته باشد ، در لیست مجاورت u مقدار $(v,5)$ ذخیره می شود.
نمایش ماتریس مجاورت :
این روش ماتریسی $|V| \times |V|$ می سازد که در آن هر ورودی $A[u][v]$ نشان دهنده وزن یال (u,v) است. اگر یال وجود نداشته باشد می توان مقدار NIL یا ∞ یا ۰ را ذخیره کرد.
ماتریس وقوع (Incidene matrix) در گراف جهت دار : یک ماتریس $|V| \times |E|$ هست

$$b_{ij} = \begin{cases} -1 & \text{if edge } j \text{ leaves vertex } i. \\ 1 & \text{if edge } j \text{ enters vertex } i. \\ 0 & \text{otherwise.} \end{cases}$$

مثال:



	e1	e2	e3	e4
A	1	1	0	0
B	-1	0	1	0
C	0	-1	-1	1
D	0	0	0	-1

دفاتر مرجان MARJAN

CamScanner

Year: Month: Date: Subject:

تمرین ۱) transpos گراف جهت دار، گرافی است که جهت یال های آن برعکس گراف اصلی است. حالا از روی G یا G^T را پیدا کن:

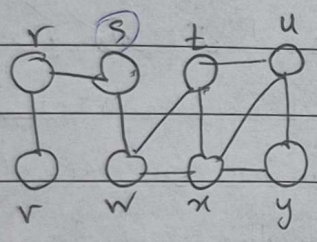
با لیست:

```
For u in adj_list:
  for v in adj_list[u]:
    Add u to adj_list_T[v]
return adj_list_T
```

با ماتریس: باید ماتریس ترانهاده کنیم (جای سطرها و ستون ها را با هم عوض کنیم)

```
for i in (1 to |V|):
  For j in (1 to |V|):
    A_T[i][j] = A[j][i]
return A_T
```

bfs: توضیحات مخصوص خودم: اول از یک راس شروع می کنیم و اون راس رو وارد صف می کنیم بعد همسایه های راس را وارد صف می کنیم و عضو اول صف را حذف می کنیم این کار را انقدر ادامه می دیدیم تا همه راس ها را ببینیم و راس تکراری هم نبینیم



$Q = [s]_0 \rightarrow Q\ [w\ r]_{1\ 1} \rightarrow [r\ t\ x]_{1\ 2\ 2} \rightarrow [t\ x\ v]_{2\ 2\ 2} \rightarrow [x\ v\ u]_{2\ 2\ 3} \rightarrow [v\ u\ y]_{3\ 3} \rightarrow [u\ y]_{3} \rightarrow [y]_{3}$

گره صفحه ۸ رو تمام بین مبدا هر راس تا s

```
Q = queue = [ ]
Q.append(s)
visited[s] = True
while Q:
  s = queue.Pop(0)
  Print(s, end = " ")
  for i in self.graph[s]:
    if visited[i] = False:
      queue.append(i)
      visited[i] = True
```

الگوریتم bfs رو توی گراف بزنیم درخت bfs ایجاد می شود

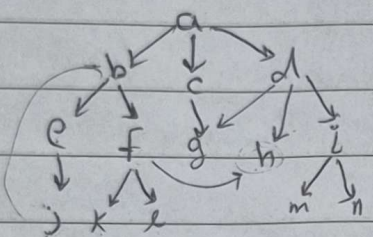
جلسه ۲۲

Year: Month: Date: Subject:

سوال: با bfs چطور میتونیم تشخیص بدیم گراف 2 بخشی است یا نه.
گراف دو بخشی اجتماع حداکثر دو زوج است. پس توی bfs میایم و هر راس رو که میبینیم رنگ مخالف با باشی میدیم اگر بتونیم برای هر راس این کار رو کنیم او کیه ولی اگر در جایی از گراف رسیدیم که color[u] == color[v] کدر با باقی u است یعنی نمی تونیم دو بخشی کنیم

سوال معروف: پیدا کردن قطر درخت: به bfs دلخواه می زنیم و دورترین راس به ریشه را در نظر میگیریم میدانیم این یک سر قطر هست. حالا روی این bfs میزنیم و جواب میشه دورترین راس از ریشه ← $O(n+m)$ که $m=n^2$ در بدترین حالت $\leftarrow O(n^2)$

dfs جستجو عمق اول:
از یه راس شروع می کنیم و به بچه های مارک نشده می رویم بچه یک درخت بررسی کردیم خلاصه.



BFS = a, b, c, d, e, f, g, h, i, j, k, l, m, n
dfs = a, b, e, j, f, k, l, h, c, g, d, i, m, n

تضمین پرانتز:
توی dfs میتونیم پرانتز گذاری صحیح داشته باشیم. یعنی موقع فاکتوری (شروع کار) (می گذاریم و موقع مسکل (کار تموم شده)) می گذاریم مثل روبین

یه گراف داریم dfs میزنیم یه درخت ایجاد میشه به اون میگیم درخت dfs و یال هاش میگیم Tree edge

back edge ← (یال های برگشتی): یال هایی که یک راس را به یکی از اجداد آن در درخت dfs متصل می کند
forward edge (یال های رو به جلو): یال هایی که به یک راس فرزند متصل شوند اما جز یال های درخت dfs نیستند این یال ها مثلا درخت dfs را شکل نداده اند اما به گونه ای می روند که در زیر شاخه درخت قرار دارد.
cross edge (یال های متقاطع) یال هایی که بین دو راس در یک زیر گراف dfs یا بین راس هایی در دو زیر گراف متفاوت dfs قرار دارند. معمولا این یال ها به راس هایی متصل می شوند که قبلا پیدا شون شده اما هیچ ارتباطی به درخت حاری dfs ندارند.

tree or forward edge → $d[u]<d[v]<f[v]<f[u]$
back edge $d[v]<d[u]<f[u]<f[v]$
cross edge $d[v]<f[v]<d[u]<f[u]$

$d[u]$ = زمان کشف راس u
$f[u]$ = زمان خاتمه پیدا کردن راس u

حتما مثال ها و تمرین های جزوه رو ببینن